OKE Access and Bastion Demo Labs

kubectl, Cloud Shell, SSH and OCI Bastion access to an OKE cluster with managed nodes

These labs use an OKE cluster with managed nodes. You can perform the demonstrations from the OCI Console, Cloud Shell, or a local terminal.

> **Production:** For production, use private worker nodes and a private Kubernetes API endpoint wherever possible.

Lab topology

Component	Example
Region	ap-hyderabad-1
Cluster	demo-oke-managed
API endpoint	Public for the basic lab; private for the Bastion lab
Worker nodes	Managed nodes
Worker subnet	Public for direct SSH lab; private for Bastion SSH lab
Bastion subnet	Private regional subnet
Kubernetes API port	TCP 6443
SSH port	TCP 22

Lab 1: Access a Cluster Using kubectl

Objective

Configure kubectl and verify access to an OKE cluster.

Prerequisites

- An active OKE cluster.
- A Kubernetes user with permission to access the cluster.
- OCI IAM permission to read the cluster.
- kubectl installed if using a local terminal.
- OCI CLI installed if using a local terminal.
- An OCI API signing key configured locally when using the OCI CLI.

Console steps

1. Sign in to the OCI Console.
2. Select region **ap-hyderabad-1**.
3. Open the navigation menu.
4. Go to **Developer Services > Containers & Artifacts > Kubernetes Clusters (OKE)**.
5. Select the compartment containing the cluster.
6. Click the cluster name.
7. Click **Access cluster**.
8. Select the appropriate access method: **Cloud Shell Access** or **Local Access**.
9. Copy or follow the displayed commands.

The Console provides a cluster-specific `oci ce cluster create-kubeconfig` command.

Generate kubeconfig

```
oci ce cluster create-kubeconfig \
  --cluster-id <cluster-ocid> \
  --file $HOME/.kube/config \
  --region ap-hyderabad-1 \
  --token-version 2.0.0
```

Replace `<cluster-ocid>` with the OCID of your cluster. If you use a different file:

```
export KUBECONFIG=$HOME/.kube/demo-oke-config
```

Set secure permissions:

```
chmod 600 $KUBECONFIG
```

Verify access

Run:

```
kubectl config current-context
kubectl get nodes
kubectl get pods -A
```

Expected result:

- The node status should be **Ready**.
- System pods should be in **Running** or an appropriate completed state.

Additional verification:

```
kubectl cluster-info
kubectl get namespaces
```

Lab 2: Access the Cluster Using Cloud Shell

Objective

Use OCI Cloud Shell without installing OCI CLI or kubectl on a local computer.

Steps

1. Open the OKE cluster details page.
2. Click **Access cluster**.
3. Select **Cloud Shell Access**.
4. Click **Launch Cloud Shell**, if shown.
5. Wait for Cloud Shell to start.
6. Run the create-kubeconfig command shown by the Console.

Typical command:

```
oci ce cluster create-kubeconfig \
  --cluster-id <cluster-ocid> \
  --file $HOME/.kube/config \
  --region ap-hyderabad-1 \
  --token-version 2.0.0
```

Verify the generated file:

```
ls -l $HOME/.kube/config
```

Test the cluster:

```
kubectl get nodes
kubectl get pods -A
```

Demonstration test

Create a temporary test deployment:

```
kubectl create deployment nginx --image=nginx
kubectl get deployment
kubectl get pods -o wide
```

Expose it temporarily:

```
kubectl expose deployment nginx \
  --port=80 \
  --type=ClusterIP
```

Verify the service:

```
kubectl get service nginx
```

Clean up the demonstration resources:

```
kubectl delete service nginx
kubectl delete deployment nginx
```

Lab 3: Access the Cluster Using a Local Terminal

Objective

Configure a workstation to access the OKE cluster.

Step 1: Install the required software

- OCI CLI
- kubectl
- An SSH client, if using Bastion or SSH labs

Verify the installations:

```
oci --version
kubectl version --client
ssh -V
```

Step 2: Configure OCI CLI

Run:

```
oci setup config
```

Provide:

- User OCID
- Tenancy OCID
- Region: ap-hyderabad-1
- API key fingerprint
- Private key location

Upload the public API key to: **Profile menu > User settings > API keys > Add API key**.

> **Important:** Do not upload or share your private key.

Step 3: Generate kubeconfig

```
mkdir -p $HOME/.kube

oci ce cluster create-kubeconfig \
  --cluster-id <cluster-ocid> \
  --file $HOME/.kube/demo-oke-config \
  --region ap-hyderabad-1 \
  --token-version 2.0.0
```

Set the active configuration:

```
export KUBECONFIG=$HOME/.kube/demo-oke-config
```

To make it persistent in Bash:

```
echo 'export KUBECONFIG=$HOME/.kube/demo-oke-config' >> ~/.bashrc
source ~/.bashrc
```

Step 4: Test local access

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info
```

Troubleshooting checks

If access fails, verify:

```
oci iam region-subscription list --tenancy-id <tenancy-ocid>
oci ce cluster get --cluster-id <cluster-ocid>
```

Also check:

- The OCI CLI profile is correct.
- The selected region is correct.
- Your IAM policies allow cluster access.
- The API endpoint is reachable from your local network.
- A private API endpoint requires a private network path or Bastion tunnel.

Lab 4: Connect to Managed Nodes Using SSH

There are two different SSH demonstrations:

- Direct SSH to nodes in a public subnet.
- SSH to private nodes through OCI Bastion.

Managed nodes support SSH when the node pool was created with an SSH public key.

> **Important:** Virtual nodes do not support SSH access. This lab applies to managed nodes only.

Lab 5: Direct SSH to Managed Nodes in a Public Subnet

Objective

Connect directly to a managed worker node that has a public IP address.

Prerequisites

- Managed nodes are in a public subnet.
- The node has a public IP address.
- An SSH public key was supplied during node-pool creation.
- The subnet security list or NSG allows TCP port 22.
- Your current public IP is allowed as the source.

Avoid using `0.0.0.0/0` for SSH in a real environment. Use your workstation's public CIDR, for example:

```
203.0.113.25/32
```

Step 1: Verify the SSH rule

In the OCI Console:

1. Open the cluster.
2. Open the node pool.
3. Open the worker node or its Compute instance.
4. Identify the VNIC and subnet.
5. Open the subnet's security list or the node VNIC's NSG.

Confirm an ingress rule exists:

Source CIDR	Protocol	Destination port
Your public IP/32	TCP	22

Step 2: Obtain the node public IP

From the OCI Console:

1. Open **Compute > Instances**.
2. Select the worker node.
3. Copy the **Public IPv4 address**.

You can also inspect nodes through Kubernetes:

```
kubectl get nodes -o wide
```

The OCI Console is the authoritative place to confirm the node's public IP and VNIC configuration.

Step 3: Connect to the node

For an Oracle Linux worker node, use the `opc` user:

```
chmod 600 ~/.ssh/oke_demo_key

ssh -i ~/.ssh/oke_demo_key opc@<node-public-ip>
```

Replace `<node-public-ip>` with the node's public IP.

Step 4: Verify the node

After connecting:

```
hostname
cat /etc/os-release
sudo crictl info
```

Note: You can inspect, but do not manually modify, OKE-managed configuration unless required by a supported procedure.

Exit the node:

```
exit
```

Cleanup

For a temporary lab:

- Remove the SSH ingress rule.
- Remove the public IP if it is no longer needed.
- Delete the node pool or cluster only after completing any required demonstrations.

Lab 6: Set Up a Bastion for Cluster Access

Objective

Use OCI Bastion to access private OKE resources without assigning public IP addresses to worker nodes.

A Bastion can be used for:

- Kubernetes API endpoint access through a port-forwarding session.
- SSH access to managed nodes through a managed SSH session.

Recommended network design

Create or use:

- A private Bastion subnet.
- A private Kubernetes API endpoint subnet.
- A private worker-node subnet.
- A NAT gateway if private nodes need outbound Internet access.
- Appropriate route tables.
- Security list or NSG rules.

The Bastion subnet must be able to reach the target private resource.

Lab 7: Configure Network Rules for Bastion

A. Kubernetes API endpoint access

For a private API endpoint, allow TCP 6443. The API endpoint subnet must allow ingress from the Bastion subnet:

Source	Protocol	Destination port
Bastion subnet CIDR	TCP	6443

The Bastion subnet must allow egress to the API endpoint:

Destination	Protocol	Destination port
API endpoint subnet CIDR	TCP	6443

B. Managed-node SSH access

For private worker nodes, allow TCP 22. The worker-node subnet or node NSG must allow ingress from the Bastion subnet:

Source	Protocol	Destination port
Bastion subnet CIDR	TCP	22

The Bastion subnet must allow egress to the worker-node subnet:

Destination	Protocol	Destination port
Worker subnet CIDR	TCP	22

> **Tip:** Use NSGs where possible to limit these rules to only the required resources.

Lab 8: Create a Bastion in the OCI Console

Steps

1. Open the navigation menu.
2. Go to **Identity & Security > Bastion**.
3. Select the correct compartment.
4. Click **Create bastion**.
5. Enter a name, for example `demo-oke-bastion`.
6. Select the target VCN, the private Bastion subnet, and an allowed client CIDR block.

> **Note:** For a lab, the allowed CIDR can be your current public IP in /32 format. Avoid using 0.0.0.0/0 unless it is strictly temporary.

7. Configure the maximum session time-to-live if required.
8. Click **Create bastion**.
9. Wait until the Bastion status becomes **Active**.

Lab 9: Set Up a Bastion to Access the Kubernetes API Endpoint

Objective

Access a private Kubernetes API endpoint through a Bastion SSH port-forwarding session.

Step 1: Obtain the API endpoint private IP

1. Open the OKE cluster.
2. Select the Resources or Cluster details area.
3. Locate the Kubernetes API endpoint.
4. Record the private IP address or private endpoint information.

Use the private API endpoint address for the Bastion target.

Step 2: Create a port-forwarding session

1. Open the Bastion details page.
2. Open the **Sessions** tab.
3. Click **Create session**.
4. Select **SSH port forwarding session**.
5. Enter a session name, for example `demo-oke-api-forward`.

Specify:

6. **Target private IP:** the Kubernetes API endpoint private IP.
7. **Target port:** 6443.
8. **SSH public key:** upload or paste the public key used for the session.
9. Set a short session duration for the lab.
10. Click **Create session**.
11. Wait until the session becomes **Active**.

Step 3: Copy the tunnel command

On the session details page:

1. Click the session action menu.
2. Select **Copy SSH command**.
3. Paste the command into Cloud Shell or your local terminal.
4. Replace the private key path if required.
5. Change the local forwarding port to 6443, if necessary.

The command generally resembles:

```
ssh -i ~/.ssh/oke_bastion_key \
  -N \
  -L 6443:<api-private-ip>:6443 \
  -p 22 <bastion-session-ocid>
```

Important: Use the exact command generated by the Console, because it contains the correct Bastion session parameters. Keep this terminal running.

Step 4: Generate or update kubeconfig

Generate kubeconfig:

```
oci ce cluster create-kubeconfig \
  --cluster-id <cluster-ocid> \
  --file $HOME/.kube/demo-oke-private-config \
  --region ap-hyderabad-1 \
  --token-version 2.0.0
```

Set the file as the active kubeconfig:

```
export KUBECONFIG=$HOME/.kube/demo-oke-private-config
```

For the Bastion tunnel, configure the Kubernetes API server address to use the local forwarded port:

```
https://127.0.0.1:6443
```

Before changing the file, make a backup:

```
cp $KUBECONFIG ${KUBECONFIG}.backup
```

Use the Console-generated Bastion instructions for the exact kubeconfig modification required by the cluster endpoint configuration.

Step 5: Test the private API connection

With the SSH tunnel still running:

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info
```

If the tunnel is working, kubectl communicates with the private API endpoint through `127.0.0.1:6443`.

Step 6: Stop the tunnel

When finished, return to the terminal running the SSH command and press **Ctrl+C**. You can also delete the Bastion session or allow it to expire.

Lab 10: Set Up Bastion Access to Managed Nodes

Objective

Connect to private managed worker nodes using an OCI Bastion managed SSH session.

Step 1: Confirm the node is managed

1. Open the OKE cluster.
2. Open **Node pools**.
3. Select the node pool.
4. Confirm the node type is **Managed**.
5. Open the worker node's Compute instance details.
6. Record the node's private IP address.

Step 2: Confirm the node has an SSH public key

The node must have been created with an SSH public key.

> **Warning:** If the node was not created with an SSH key, do not assume that an existing key can be added safely to the managed node. Use a new node pool or follow an approved OKE node-access procedure.

Step 3: Create a managed SSH session

1. Open **Identity & Security > Bastion**.
2. Open the Bastion resource.
3. Select **Sessions**.
4. Click **Create session**.
5. Select **Managed SSH session**.
6. Enter a session name, for example `demo-oke-node-ssh`.
7. Select the target Compute instance corresponding to the managed node.
8. Provide the SSH public key.
9. Select the appropriate username, normally `opc`.
10. Set the session TTL.
11. Click **Create session**.
12. Wait until the session becomes **Active**.

Step 4: Copy the SSH command

1. Open the active session.
2. Click **Copy SSH command**.
3. Paste the command into Cloud Shell or your local terminal.
4. Replace the private key path if necessary.
5. Run the command.

A typical command resembles:

```
ssh -i ~/.ssh/oke_node_key \
  -o ProxyCommand="ssh -i ~/.ssh/oke_bastion_key -W %h:%p -p 22 <bastion-session-ocid>" \
  opc@<node-private-ip>
```

Note: The command generated by the Console is authoritative for the session. Use the exact command from the Bastion Console rather than constructing it manually.

Step 5: Verify node access

After connecting:

```
hostname
hostname -I
cat /etc/os-release
```

Optional inspection commands:

```
sudo systemctl status kubelet
sudo crictl ps
```

Exit:

```
exit
```

Lab 11: Complete End-to-End Demonstration

Use the following sequence for a classroom or workshop demonstration.

Part 1: Cluster access

1. Open the OKE cluster.
2. Click **Access cluster**.
3. Launch Cloud Shell.
4. Generate kubeconfig.
5. Run:

```
kubectl get nodes
kubectl get pods -A
```

Part 2: Kubernetes test deployment

```
kubectl create deployment demo-nginx --image=nginx
kubectl get pods -o wide
kubectl expose deployment demo-nginx --port=80 --type=ClusterIP
kubectl get service demo-nginx
```

Clean up:

```
kubectl delete service demo-nginx
kubectl delete deployment demo-nginx
```

Part 3: Direct SSH demonstration

1. Use a managed node in a public subnet.
2. Confirm TCP 22 is allowed only from your IP.
3. Copy the node public IP.
4. Connect using:

```
ssh -i ~/.ssh/oke_node_key opc@<node-public-ip>
```

5. Run `hostname`.
6. Exit.

Part 4: Bastion API demonstration

1. Use a cluster with a private API endpoint.
2. Create the Bastion.
3. Create a port-forwarding session to the API endpoint on port 6443.
4. Copy and run the SSH tunnel command.
5. Use the Bastion-specific kubeconfig configuration.
6. Run `kubectl get nodes`.
7. Stop the tunnel.

Part 5: Bastion managed-node demonstration

1. Use a managed node in a private subnet.
2. Confirm Bastion-to-node TCP 22 rules.
3. Create a managed SSH session.
4. Copy and run the generated SSH command.
5. Run `hostname`.
6. Exit and allow the session to expire or delete it.

Common Problems and Checks

kubectl cannot connect

- Kubeconfig points to the correct cluster.
- KUBECONFIG points to the expected file.
- The OCI CLI region is correct.
- The cluster API endpoint is reachable.
- A Bastion tunnel is running for a private endpoint.
- The local kubeconfig server address matches the tunnel address.

Bastion session cannot be created

- Bastion is in the correct compartment.
- The target IP is private and reachable from the Bastion subnet.
- The Bastion allowlist includes your current public IP.
- IAM permissions allow Bastion session creation.
- The target port is correct: 6443 for Kubernetes API or 22 for SSH.

SSH connection fails

- The node is a managed node.
- The correct private key matches the public key on the node.
- The username is opc.
- The worker subnet allows TCP 22 from the Bastion subnet.
- The node private IP is correct.
- The Bastion session is still active.
- Private key permissions are restricted:

```
chmod 600 ~/.ssh/oke_node_key
chmod 600 ~/.ssh/oke_bastion_key
```

Do not use

Avoid:
• A private SSH key in the OCI Console.
• 0.0.0.0/0 for permanent SSH access.
• Public IP addresses for production worker nodes unless explicitly required.
• Manual changes to OKE-managed node configuration without a supported procedure.
• A virtual node for an SSH demonstration.

Sources

https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengdownloadkubeconfigfile.htm
https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengsettingupbastion.htm
https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengconnectingworkernodesusingssh.htm
https://docs.oracle.com/en-us/iaas/Content/Bastion/Tasks/create-bastion.htm
https://docs.oracle.com/en-us/iaas/Content/Bastion/Tasks/connectingtosessions.htm
https://docs.oracle.com/en-us/iaas/Content/Bastion/Concepts/bastionoverview.htm